Simulation of 2D Elastic Collision: A Force-Based Physics Engine

Physics Grade 11A Project - Group 5 and 10
St John Baptist De La Salle Catholic School

March 20, 2026

Project Resources

- **Source Code (GitHub):** `github.com/BOSSNATY/2D-Elastic-Collision-Engine`
- **Beamer Presentation:** `2D Elastic Collision Presentation`

1 Introduction

Building a physics engine from scratch is a complex task that involves simulating the fundamental mechanics of the physical world. The goal of this project was to move beyond simple velocity-swapping shortcuts; we wanted to simulate the "why" behind a bounce. By treating every collision as a microscopic interaction between springs, the engine applies Newtonian mechanics and numerical integration to resolve motion in real-time.

2 The Physics Framework

The core of the simulation relies on two fundamental laws of classical mechanics: **Hooke's Law** and **Newton's Second Law**. In this framework, objects are not treated as perfectly rigid; rather, they compress and push back upon impact.

2.1 Force Generation

Pucks are simulated as having internal stiff springs. When they overlap, a restorative force (F) is generated according to Hooke's Law:

$$F = k \cdot \text{overlap} \tag{1}$$

where k represents the stiffness constant.

2.2 Acceleration

Once the force is determined, Newton's Second Law is used to calculate the resulting motion. The mass (m) of the puck dictates its resistance to acceleration:

$$a = \frac{F}{m} \tag{2}$$

3 Mathematical Approach and Numerical Integration

Because computers process time in discrete snapshots rather than a continuous flow, the **Euler Method** is employed for numerical integration. This allows the engine to predict motion in tiny time increments (dt).

3.1 The Collision Resolution Loop

Every iteration of the simulation follows these steps:

1. **Detection:** Monitor the distance (d) between centers. A collision occurs if $d < (r_1 + r_2)$.
2. **Overlap Calculation:**
$$\text{Overlap} = (r_1 + r_2) - d \tag{3}$$
3. **Directional Vectors:** Calculate the unit vector ($\hat{n}$) to determine the direction of the force.
4. **Integration:** Update velocity and position using the time step dt:
$$v_{new} = v_{old} + (a \cdot dt) \tag{4}$$
$$x_{new} = x_{old} + (v_{new} \cdot dt) \tag{5}$$

4 Challenges and Debugging

- **Numerical Instability:** If k is too high or dt is too large, the math can go high, causing objects to gain infinite energy. Finding the goldilocks zone for these values was a major part of debugging.
- **Energy Drift:** As an approximation, the Euler Method can accidentally add extra speed, potentially violating the law of conservation of energy.

5 Algebraic Implementation

For two pucks (1 and 2), the complete mathematical flow is defined as follows:

Spatial Calculations

$$dx = x_2 - x_1 \tag{6}$$
$$dy = y_2 - y_1 \tag{7}$$
$$d = \sqrt{dx^2 + dy^2} \tag{8}$$
$$\text{Overlap} = (r_1 + r_2) - d \tag{9}$$

Vector and Force Components

$$n_x = \frac{dx}{d}, \quad n_y = \frac{dy}{d} \tag{10}$$

$$F_{mag} = k \cdot \text{Overlap} \tag{11}$$

$$F_x = F_{mag} \cdot n_x, \quad F_y = F_{mag} \cdot n_y \tag{12}$$

Acceleration and Motion Update

For Puck 1:

$$a_{x1} = -\frac{F_x}{m_1}, \quad a_{y1} = -\frac{F_y}{m_1} \tag{13}$$

$$v_{x1_new} = v_{x1_old} + (a_{x1} \cdot dt) \tag{14}$$

$$x_{1_new} = x_{1_old} + (v_{x1_new} \cdot dt) \tag{15}$$

For Puck 2:

$$a_{x2} = \frac{F_x}{m_2}, \quad a_{y2} = \frac{F_y}{m_2} \tag{16}$$

$$v_{x2_new} = v_{x2_old} + (a_{x2} \cdot dt) \tag{17}$$

$$x_{2_new} = x_{2_old} + (v_{x2_new} \cdot dt) \tag{18}$$

6 Simulation Results and Evidence

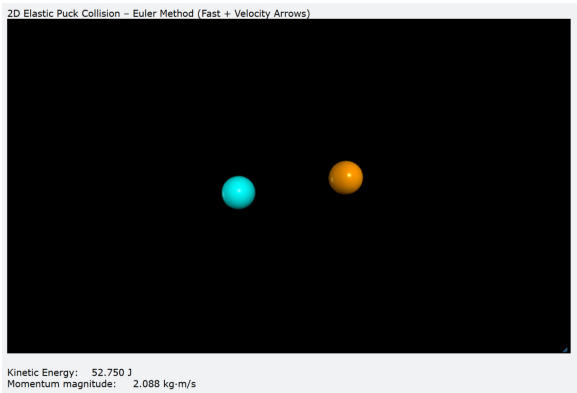

Figure 1: VPython Simulation of the 2D collision.

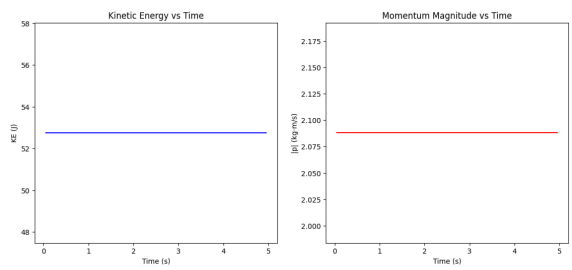

Figure 2: Graph showing conservation of Energy and Momentum.

7 Conclusion

This force-based approach effectively mirrors real-world physics by transitioning from static geometry to a dynamic system governed by Newtonian laws, providing a robust foundation for 2D physical simulations.